

Smart Mobile Phone Price Prediction Using Machine Learning

Team Name: THE BOYS

Team Members

Naga Sriram Panithapu
Hari Purna Sai Sushruth Pedapudi

College Mentor: Mrs. D Kalpana

Industrial Mentor: Mr. Srivatsa Sinha

Introduction

The "Smartphone Price Prediction Using Machine Learning" project aims to develop an intelligent system that can predict smartphone prices based on technical specifications and other related features. In an era of rapidly evolving technology where new models are frequently introduced, reliable price prediction models help consumers, manufacturers and retailers make informed decisions. This project uses machine learning algorithms to identify patterns and relationships between various attributes of smartphones, such as price range, battery power, Bluetooth, dual sim, processor clock speed, network type, touch screen, screen size, mobile weight, cores, pixels, talk time, RAM, internal storage and Wi-Fi. Data is collected from various authoritative sources, carefully pre-processed and divided into subsets for training, validation and testing. Several machine learning models are examined to identify the best performing model in terms of accuracy and versatility. The project aims to develop a robust, reliable and accurate model that can predict smartphone prices, bridging the gap between technical specifications and market price trends. Few applications of this project are for Consumer Aid, Retailer and E- Commerce, Manufacturers, Market Analysis. Although it doesn't give the actual numerical price value for the prediction, it categorizes or classifies the mobile price which it predicts. On social media, many people freely share their opinions and ratings of products. In this research, historical sentiments of people regarding price (whether it is considered high, medium, or low) are taken into account. In addition to important factors like display, processor, and memory, other features such as camera quality, thickness, battery life, and connectivity are considered to determine the approximate price of a phone. To predict the price, a dataset from Kaggle, a popular platform for datasets, is used. The dataset consists of separate CSV files for training and testing. The training dataset is used to train the model and determine which model achieves the highest accuracy. The trained model is then applied to the test dataset to predict the price.

Literature Survey

The paper "Prediction of Phone Prices Using Machine Learning Techniques" discusses the importance of considering various factors when purchasing a smartphone, including features like display, processor, memory, camera, and battery. It highlights the need for assessing whether a smartphone is worth its cost. To address this, the paper proposes using historical data on smartphone features and prices to develop a model for predicting the approximate price of a new smartphone accurately. The data set used considers 21 parameters, and algorithms such as random forest classifier, support vector machine, and logistic regression are employed. The prediction of prices helps customers make informed decisions and assists phone owners in setting appropriate pricing for their products. Support Vector Machine, with an accuracy of 81%, is used for price prediction.

Data Description

The data set was downloaded from Kaggle. As the preprocessed data were already available no preprocessing has been done on the data set. The data set has the following columns. Each of this is an attribute associated with phone. The test data set that has the same attributes as the train dataset except the price column is not available. This data set takes into account all the parameters associated with a smartphone as they are all a deciding factor on the price of the phone. Listed below are the various names of the column in the table.

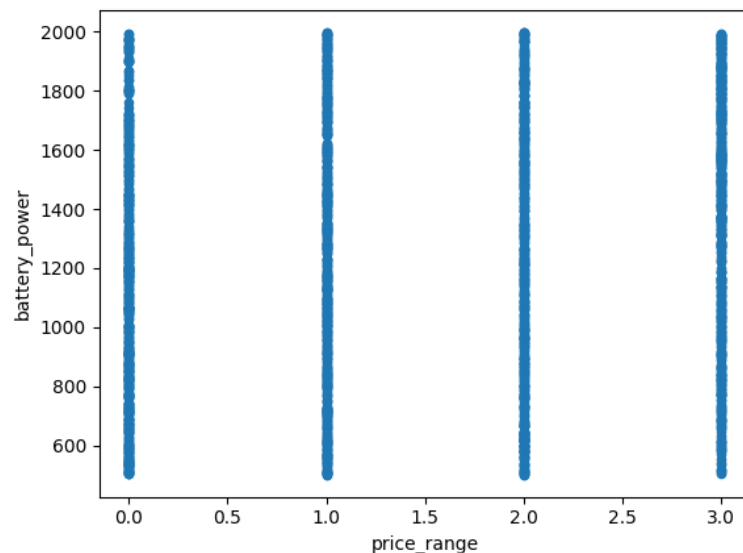
Battery_power resembles total energy that a battery can store at a time (measured in mAh), Blue tells if phone has Bluetooth or not, Clock_speed shows speed at which the microprocessor in the phone executes the instructions, Dual_sim will state if dual-sim support is offered or not, Fc is Front camera which shows focus in megapixels, Four_g represents if 4G is available or not, Int_memory shows the internal memory of phone

in gigabytes, M_dep is the mobile depth in cm, Mobile_wt tells weight of the phone, N_cores shows the number of cores in the processor, Pc resembles the primary camera focus in megapixels, Px_height states pixel resolution height, Px_width tells pixel resolution width, Ram shows the Random access memory (megabytes), Sc_h is the screen height of phone in cm, Sc_w states screen width of phone in cm, Talk_time shows the maximum time that a single battery charge will last when used for Talking, Three_g resembles whether phone has 3G or not, Touch_screen is whether the display touch screen or not, Wi-Fi states whether Wi-Fi connectivity present or not, Price_range shows the target variable of this data set. This has four discrete values of 0(low cost), 1(medium cost), 2(high cost) and 3(very high cost).

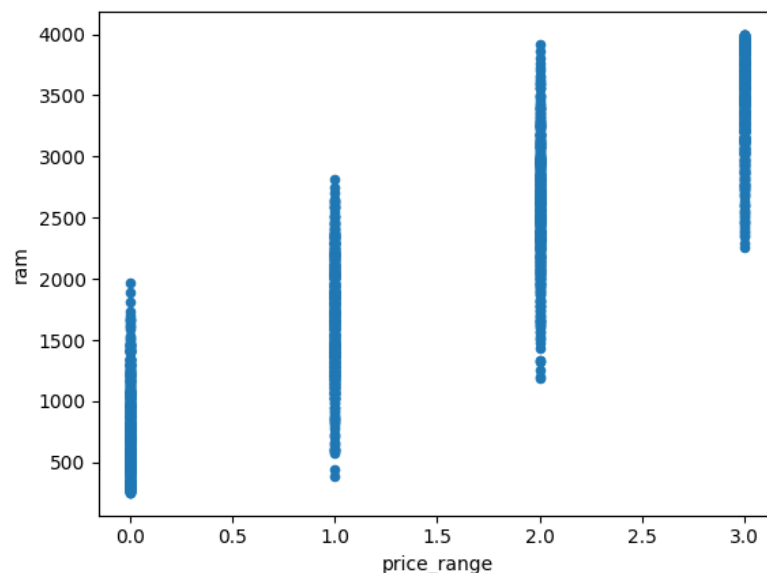
Here, the approximate price range is predicted. We use discrete value for the price in the place of exact price. The price of the column is predicted based on 21 different parameters. The data set contains data of close to 2000 odd phones.

Data Visualization

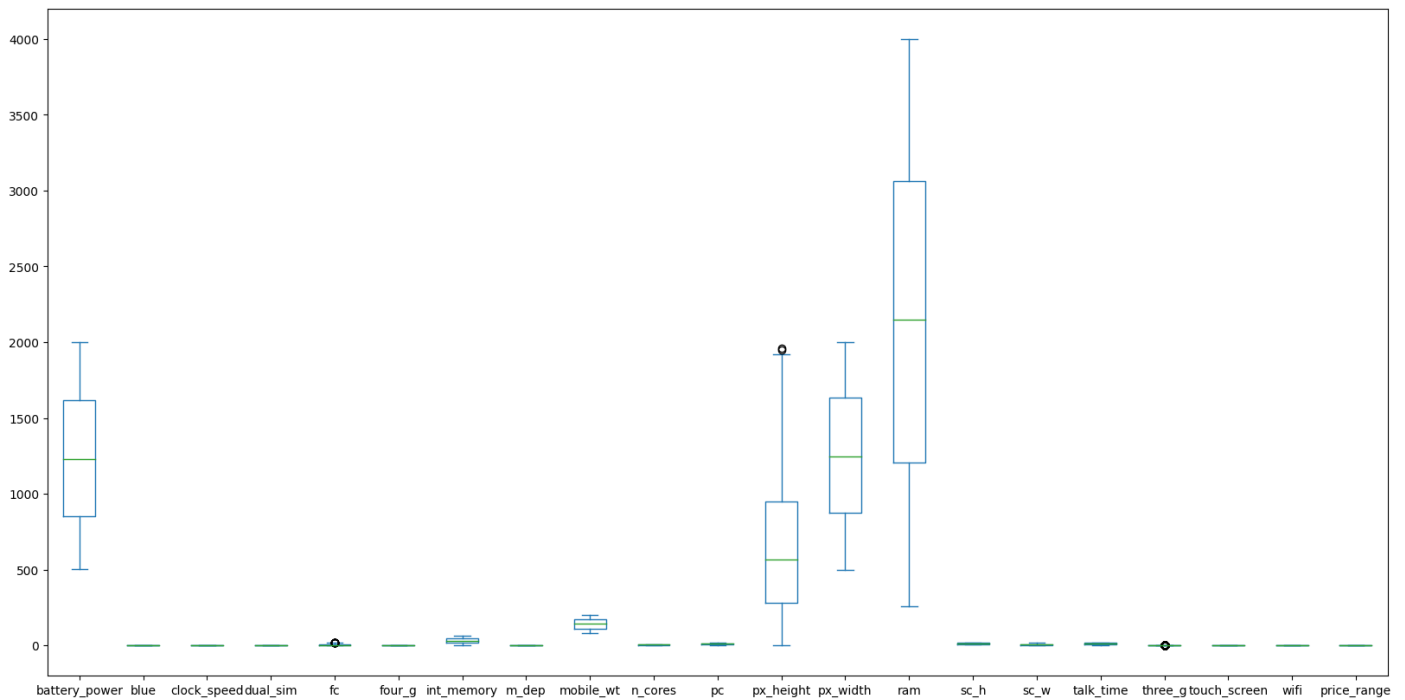
- **Price Range vs Battery**



- **Price Range vs RAM**



- **Overview**



Working

We utilized a dataset sourced from Kaggle, consisting of two files: 'train.csv' and 'test.csv'. The 'train.csv' file contained 2000 rows of labeled data for training purposes, while the 'test.csv' file contained 1000 rows of unlabeled data for testing. In our initial approach, we trained various models using the 'train.csv' dataset. Subsequently, these models were employed to predict the prices for the 'test.csv' dataset, providing valuable insights into their performance. Additionally, we employed the same model on the 'train.csv' dataset, employing the `train_test_split()` method to split the 2000 labeled data rows into 1600 rows for labeled training data and 400 rows for labeled testing data. This approach allowed us to assess the accuracy of the model by comparing the labeled target values with the predicted target values, thus evaluating its performance. By adopting these methodologies, we aimed to maximize the predictive capabilities of our models and gain insights into their accuracy and performance metrics.

Algorithms

- **Decision Tree Classifier**

Decision trees are widely used for classification and regression tasks, representing features, decisions, and class labels. Optimal splits are determined using the Gini index, measuring node impurity and aiding in data partitioning. The index ranges from 0 to 1, with lower values indicating better separability. Minimizing the Gini index enhances accuracy. This index guides feature splits, segmenting data and creating branches. It is crucial for optimal decision tree models, widely adopted in machine learning for its effectiveness.

Accuracy: 0.845

$$Gini = 1 - \sum_{i=1}^C (p_i)^2$$

- **K Neighbors Classifier**

K-nearest neighbors (KNN) is a widely used classification algorithm. It uses K nearest neighbors and default parameters like K and distance metric. K choice impacts performance, with small K values sensitive to noise and large K values causing over smoothing. Distance metric measures similarity, typically Euclidean. The classifier stores the entire training dataset and computes distances for prediction. Majority voting among K neighbors determines class label for new data points.

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Accuracy: 0.9125

- **Logistic Regression**

Logistic regression is widely used for binary classification. It models the feature-probability relationship. Default parameters control overfitting (regularization) and optimization termination (convergence). Regularization types include L1 (Lasso) and L2 (Ridge). The classifier uses the logistic function as the default activation. During training, optimization techniques minimize loss and maximize likelihood. Parameter selection and tuning are crucial for optimal binary classification.

Accuracy: 0.96

- **Gaussian Naive Bayes**

The Gaussian Naive Bayes classifier is a widely used machine learning algorithm for classification. It assumes feature independence and applies Bayes' theorem for calculating class probabilities. Default parameters include prior probabilities, representing initial beliefs, usually set uniformly across classes. Another default parameter is the variance of the Gaussian distribution, indicating data variability within each class. The classifier assumes feature values within each class follow a Gaussian distribution, simplifying computation of class probabilities. During training, the classifier estimates mean and variance for each feature in each class based on training data, facilitating likelihood calculation using the Gaussian distribution.

Accuracy: 0.7575

- **Support Vector Classifier**

The SVM classifier separates data using an optimal hyperplane and has default parameters. The kernel function transforms data, with RBF being the default. Regularization (C) balances margin and misclassifications. Default decision function labels data based on hyperplane position. During training, optimal hyperplane values are estimated via quadratic optimization to maximize margin and minimize errors. Parameter selection and tuning are important for optimal performance.

Accuracy: 0.9625

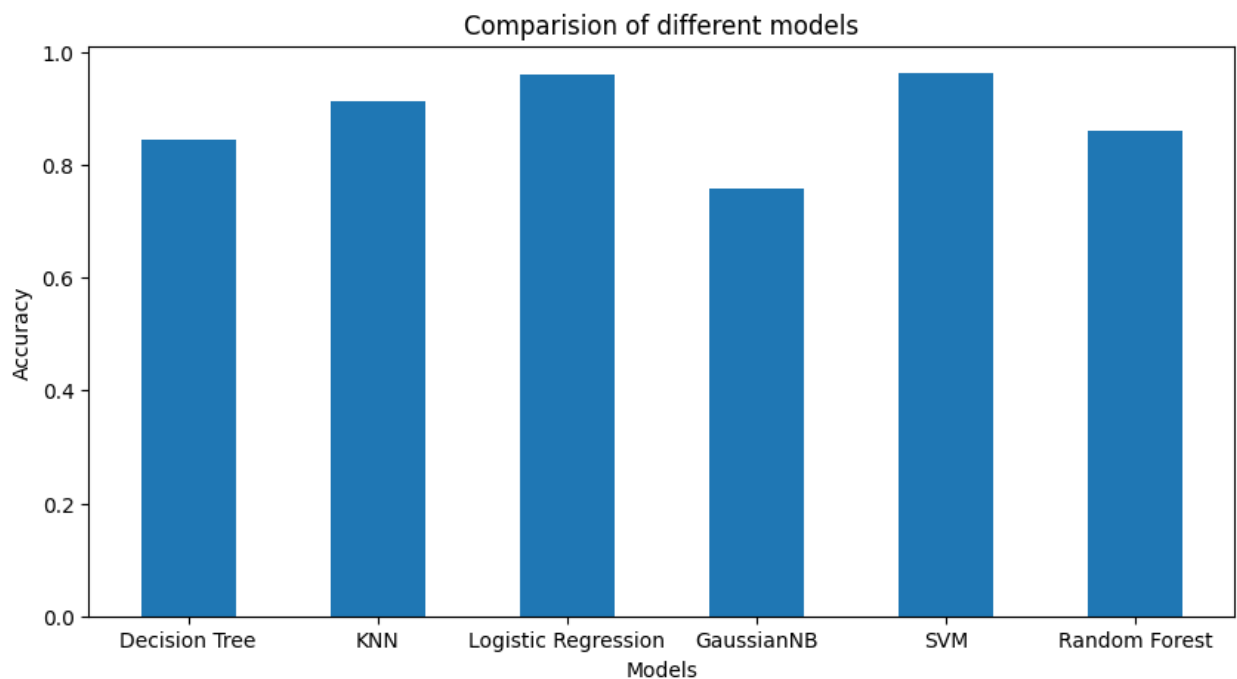
- **Random Forest Classifier**

The Random Forest classifier combines decision trees and has default parameters. One parameter is the number of trees (n_estimators), typically set to 100. Another parameter is the splitting criterion, commonly Gini impurity. Feature selection involves considering a square root of total features to reduce overfitting. Training involves bootstrap sampling and random feature subsets. Proper parameter selection is important for optimal classification results.

Accuracy: 0.86

Models	Accuracy
Decision Tree	0.845
K-Nearest Neighbor	0.9125
Logistic Regression	0.96
Gaussian Naive Bayes	0.7575
Support Vector Machine	0.9625
Random Forest	0.86

Table 1: List of Models and their Accuracies



Conclusion

In conclusion, our project on smart mobile phone price prediction using machine learning has successfully explored and compared the performance of six different algorithms. Through rigorous experimentation and analysis, we evaluated the effectiveness and accuracy of these algorithms in predicting mobile phone prices. The SVM is found to give the most accuracy of 96.25% followed by Logistic Regression of 96%. Then, the prediction of the price is done using SVM. The Gaussian Naive Bayes is found to give the least accuracy score of 75.75%.